PROYECTO INTEGRADOR · PROGRAMACIÓN

Introducción al lenguaje Dart

Variables, operadores, condicionales, ciclos, listas y diccionarios, con ejemplos del POS de restaurante

Al terminar esta guía podrás:

- Declarar variables con el tipo de dato adecuado.
- Usar operadores para hacer cálculos de caja y ventas.
- Tomar decisiones con if, else, el operador ternario y switch.
- Repetir instrucciones con for, while y do-while.
- Guardar y recorrer datos en listas y diccionarios (Map).
- Resolver 5 problemas reales del restaurante.

Contenido

1. Primeros pasos
2. Variables y tipos de datos
3. Operadores
4. Null safety
5. Condicionales
6. Listas
7. Diccionarios (Map)
8. Ciclos
9. Errores comunes
10. Ejercicios del restaurante

1. Primeros pasos

Para practicar no hace falta instalar nada: abran **dartpad.dev** en el navegador, borren el código de ejemplo, peguen el suyo y presionen **Run**. El resultado aparece en la consola de la derecha.

Estructura mínima de un programa en Dart

```
void main() {
  // Esto es un comentario: Dart no lo ejecuta
  print('Bienvenidos a La Buena Mesa');
}
```

- Todo el código de esta guía va **dentro de las llaves de void main() { }**, que es el punto donde empieza el programa. Por ahora tómenlo como un molde fijo; más adelante verán qué es una función.
- **print()** muestra un texto en la consola.
- Cada instrucción termina con **punto y coma (;)**.
- Los comentarios empiezan con ***//*** y sirven para explicar el código.

2. Variables y tipos de datos

Una **variable** es un espacio con nombre donde se guarda un dato. En Dart, cada variable tiene un **tipo** que indica qué clase de dato puede guardar.

Tipo	Qué guarda	Ejemplo en el restaurante
int	Números enteros	Cantidad de platos, precio en pesos, número de mesa
double	Números con decimales	Porcentaje de propina (0.10), promedio de ventas
String	Texto (entre comillas)	Nombre del producto, medio de pago, observaciones
bool	Verdadero (true) o falso (false)	¿La caja está abierta? ¿El cliente es frecuente?

```
void main() {
  String nombre = 'La Buena Mesa';
  int numeroMesas = 12;
  double porcentajePropina = 0.10;
  bool cajaAbierta = false;

  var ciudad = 'Bogotá';            // Dart deduce que es String
  final horaApertura = '07:00';     // se asigna una sola vez
  const int baseMaxima = 10000000;  // constante: nunca cambia

  numeroMesas = 14;                 // permitido
  // ciudad = 25;                   // ERROR: ciudad es String
  // horaApertura = '08:00';        // ERROR: es final

  print('Restaurante: $nombre en $ciudad');
  print('Mesas: $numeroMesas');
  print('Propina: ${porcentajePropina * 100} %');
  print('¿Caja abierta? $cajaAbierta');
}
```

Para recordar

• **var**: Dart deduce el tipo; el valor puede cambiar, pero el tipo no.

• **final**: el valor se asigna una sola vez (por ejemplo, la hora real de apertura).

• **const**: el valor se conoce desde que se escribe el programa y nunca cambia (por ejemplo, la base máxima de caja).

• **Interpolación**: **$variable** inserta el valor dentro de un texto; para una operación se usa **${ }**, por ejemplo **${precio * 2}**.

• **Nombres**: se escriben en *lowerCamelCase* (baseInicial, totalVenta) y deben decir qué guardan.

3. Operadores

Tipo	Operadores	Ejemplo
Aritméticos	+ - * / ~/ %	total ~/ 7 (división entera), total % 7 (residuo)
Asignación	= += -= *= ++ --	total += 6000; cantidad++;
Comparación	== != > < >= <=	total >= 150000
Lógicos	&& (y) || (o) ! (no)	rol == 'cajero' && !cajaAbierta

```dart
void main() {
  int total = 87000;
  int recibido = 100000;
  int cambio = recibido - total;
  print('Cambio: $cambio');                    // Cambio: 13000

  // Dividir la cuenta entre 7 personas
  int personas = 7;
  print(total / personas);                     // 12428.57142857143 (con / siempre da double)
  print(total ~/ personas);                    // 12428 (división entera)
  print(total % personas);                     // 4 (lo que sobra)

  total += 6000;                               // se agregó una limonada
  print('Nuevo total: $total');                // Nuevo total: 93000

  int propina = total * 10 ~/ 100;             // 10 % sin decimales
  print('Propina: $propina');                  // Propina: 9300

  String rol = 'cajero';
  bool cajaAbierta = false;
  bool puedeAbrir = (rol == 'cajero' || rol == 'admin') && !cajaAbierta;
  print('¿Puede abrir la caja? $puedeAbrir');  // true
}
```

Dinero sin decimales

En pesos colombianos no se manejan centavos. Para calcular un porcentaje sin decimales, multipliquen primero y dividan después con **~/**: **total * 10 ~/ 100**. Si hacen **total * 0.10**, el resultado será un double (por ejemplo, 9300.0).

4. Null safety: datos que pueden faltar

En Dart una variable **no puede quedar vacía (null)**, a menos que su tipo lleve **?**. Esto evita uno de los errores más comunes en programación: usar un dato que no existe.

```dart
void main() {
  String? observacion;                         // puede ser null; empieza en null
  print(observacion ?? 'Sin observaciones');    // ?? usa un valor por defecto

  observacion = 'Sin cebolla';
  print('Nota para cocina: $observacion');

  String? mesero;
  mesero ??= 'Sin asignar';                    // asigna solo si es null
  print('Mesero: $mesero');

  Map<String, int> precios = {'Ajiaco': 28000};
  int? precioCazuela = precios['Cazuela'];     // no existe: da null
  print(precioCazuela ?? 'Producto no encontrado');

  int precioAjiaco = precios['Ajiaco']!;       // ! significa: "estoy seguro de que existe"
  print(precioAjiaco);
}
```

Usen **!** solo cuando estén seguros de que el dato existe; si se equivocan, el programa se detiene con un error.

5. Condicionales

5.1 if, else if y else

```
void main() {
  int baseInicial = 150000;

  if (baseInicial < 0) {
    print('Error: la base no puede ser negativa');
  } else if (baseInicial > 10000000) {
    print('Error: la base supera el máximo permitido');
  } else {
    print('Caja abierta con una base de $baseInicial');
  }
}
```

5.2 Operador ternario

Es un if/else corto que devuelve un valor: **condición ? valorSiEsVerdad : valorSiEsFalso**.

```
void main() {
  int total = 162000;
  String tipoVenta = total >= 150000 ? 'Venta grande' : 'Venta normal';
  print(tipoVenta);                              // Venta grande

  bool incluirPropina = false;
  int propina = incluirPropina ? total * 10 ~/ 100 : 0;
  print('Propina: $propina');                    // Propina: 0
}
```

5.3 switch

Útil cuando una variable puede tomar varios valores conocidos, como el medio de pago.

```
void main() {
  String medioPago = 'debito';

  switch (medioPago) {
    case 'efectivo':
      print('Calcular el cambio');
    case 'debito':
      print('Registrar la referencia del datáfono');
    case 'credito':
      print('Verificar que el cliente esté registrado');
    default:
      print('Medio de pago no válido');
  }
}
```

En Dart 3 (la versión actual) no es necesario escribir **break** al final de cada caso. **default** se ejecuta cuando ningún caso coincide.

6. Listas (List)

Una **lista** guarda varios datos en orden. Cada dato tiene una **posición (índice)** que empieza en **0**.

```
void main() {
  List<String> pedido = ['Ajiaco', 'Limonada'];

  pedido.add('Postre del día');               // agrega al final
  pedido.insert(0, 'Empanadas');              // agrega en la posición 0
  print(pedido);              // [Empanadas, Ajiaco, Limonada, Postre del día]

  print(pedido[0]);                           // Empanadas
  print(pedido.length);                       // 4
  print(pedido.contains('Limonada'));         // true
  print(pedido.indexOf('Ajiaco'));            // 1

  pedido.remove('Limonada');                  // elimina por valor
  pedido.removeAt(0);                         // elimina por posición
  print(pedido);                              // [Ajiaco, Postre del día]

  List<int> ventasDelDia = [45000, 120000, 38000];
  ventasDelDia.add(76000);
  print(ventasDelDia.first);                  // 45000
  print(ventasDelDia.last);                   // 76000
  print(ventasDelDia.isEmpty);                // false
}
```

Operación	Qué hace
lista[i]	Lee el elemento de la posición i (empieza en 0).
add(x) / insert(i, x)	Agrega al final / en la posición i.
remove(x) / removeAt(i)	Elimina por valor / por posición.
length, isEmpty, isNotEmpty	Cantidad de elementos y si está vacía o no.
contains(x), indexOf(x)	Si contiene x y en qué posición está (-1 si no está).
first, last	Primer y último elemento.

7. Diccionarios (Map)

Un **Map** guarda pares **clave: valor**. Es ideal para el menú (producto: precio) o un pedido (producto: cantidad). Las claves no se repiten.

```
void main() {
  Map<String, int> menu = {'Ajiaco': 28000, 'Limonada': 6000};

  menu['Bandeja paisa'] = 32000;              // agregar
  menu['Limonada'] = 6500;                    // modificar
  print(menu['Ajiaco']);                      // 28000
  print(menu.containsKey('Sancocho'));        // false

  menu.remove('Limonada');                    // eliminar
  print(menu.keys);                           // (Ajiaco, Bandeja paisa)
  print(menu.values);                         // (28000, 32000)
  print(menu.length);                         // 2

  for (var item in menu.entries) {            // recorrer clave y valor
    print('${item.key}: ${item.value}');
  }
}
```

¿Por qué menu['Ajiaco'] puede necesitar !?

Cuando se lee una clave, Dart no sabe si existe, así que el resultado es **int?** (puede ser null). Para operar con él, aseguren que existe con **containsKey** y luego usen **!**: **int precio = menu['Ajiaco']!;**

Combinando Map y List

```
void main() {
  Map<String, List<String>> carta = {
    'Platos fuertes': ['Ajiaco', 'Bandeja paisa'],
    'Bebidas': ['Limonada', 'Gaseosa'],
  };

  carta['Bebidas']!.add('Jugo natural');

  for (var categoria in carta.keys) {
    print('$categoria: ${carta[categoria]}');
  }
  // Platos fuertes: [Ajiaco, Bandeja paisa]
  // Bebidas: [Limonada, Gaseosa, Jugo natural]
}
```

8. Ciclos (estructuras de repetición)

8.1 for clásico (cuando importa la posición)

```
void main() {
  List<String> estados = ['libre', 'ocupada', 'libre', 'en cuenta', 'ocupada'];

  for (var i = 0; i < estados.length; i++) {
    print('Mesa ${i + 1}: ${estados[i]}');
  }
}
```

8.2 for-in (para recorrer todos los elementos)

```
void main() {
  List<int> ventas = [45000, 120000, 38000, 76000];
  int total = 0;

  for (var venta in ventas) {
    total += venta;
  }
  print('Total vendido: $total');              // Total vendido: 279000
}
```

8.3 while (mientras se cumpla una condición)

```
void main() {
  List<String> pendientes = ['Ajiaco', 'Sancocho', 'Limonada'];

  while (pendientes.isNotEmpty) {
    String plato = pendientes.removeAt(0);
    print('Preparando $plato... quedan ${pendientes.length}');
  }
  print('Todos los pedidos están listos');
}
```

8.4 do-while (se ejecuta al menos una vez)

```
void main() {
  List<String> intentos = ['1111', '2222', '5831'];
  String codigoCorrecto = '5831';
  int i = 0;
  bool acceso = false;

  do {
    print('Intento ${i + 1}: código ${intentos[i]}');
    if (intentos[i] == codigoCorrecto) {
      acceso = true;
    }
    i++;
  } while (!acceso && i < intentos.length);

  print(acceso ? 'Acceso concedido' : 'Acceso denegado');
}
```

8.5 break y continue

```
void main() {
  Map<String, int> disponibles = {'Ajiaco': 5, 'Sancocho': 0, 'Bandeja paisa': 3};

  for (var plato in disponibles.keys) {
    if (disponibles[plato] == 0) {
      print('$plato: agotado');
      continue;                                  // salta al siguiente plato
    }
    print('$plato: hay ${disponibles[plato]}');
  }

  List<int> ventas = [45000, 120000, 38000];
  for (var venta in ventas) {
    if (venta > 100000) {
      print('Primera venta grande: $venta');
      break;                                     // termina el ciclo
    }
  }
}
```

9. Errores comunes

Error	Ejemplo	Solución
Olvidar el punto y coma	print('Hola')	print('Hola');
Asignar un tipo distinto	int total = '5000';	int total = 5000;
Índice fuera de rango	lista[3] en una lista de 3 elementos	Las posiciones van de 0 a length - 1.
Operar con un valor que puede ser null	menu['Ajiaco'] * 2	menu['Ajiaco']! * 2 (después de verificar con containsKey)
Asignar un double a un int	int propina = total * 0.10;	int propina = total * 10 ~/ 100;
Confundir = con ==	if (rol = 'admin')	if (rol == 'admin')
Ciclo infinito	while (i < 5) { } sin aumentar i	Asegurar que la condición deje de cumplirse (i++).

10. Ejercicios del restaurante

Los cinco ejercicios simulan una jornada del POS, de menor a mayor dificultad. Copien los **datos iniciales** en DartPad y escriban debajo el código que resuelve el problema, siempre dentro de **void main()**. Con los datos dados, su programa debe mostrar el **resultado esperado**. Después cambien los datos para probar los demás casos.

#	Ejercicio	Practica
1	Abrir la caja	Variables, operadores lógicos, if/else, listas
2	Registrar la venta de una mesa	Map, for-in, continue, ternario, switch
3	Venta con descuento	Acumuladores, condicionales anidados, switch, null safety
4	Devolución de un producto	Validaciones, modificar y eliminar en un Map

#	Ejercicio	Practica
5	Jornada completa y cierre de caja	Listas paralelas, for con índice, totales por medio de pago

Todos los ejercicios usan este menú (cópienlo donde se indique):

Menú del restaurante

```
Map<String, int> menu = {
  'Ajiaco': 28000,
  'Bandeja paisa': 32000,
  'Sancocho': 26000,
  'Limonada': 6000,
  'Jugo natural': 7000,
  'Gaseosa': 5000,
  'Postre del día': 9000,
};
```

Ejercicio 1 · Abrir la caja

Situación: al empezar el día, el cajero debe abrir la caja con una base de dinero para dar cambio. El sistema debe validar las reglas antes de permitirlo.

Datos iniciales

```
void main() {
  String usuario = 'Laura';
  String rol = 'cajero';           // puede ser 'admin', 'cajero' o 'mesero'
  bool cajaAbierta = false;
  int baseInicial = 200000;
  const int baseMaxima = 10000000;
  List<String> movimientos = [];

  // Escriba aquí su código
}
```

Reglas que debe programar:

1. Si la caja ya está abierta, mostrar: “Error: ya hay una caja abierta”.
2. Solo los roles **cajero** y **admin** pueden abrir la caja.
3. La base no puede ser negativa ni mayor que baseMaxima.
4. Si todo está bien: cambiar cajaAbierta a true, agregar a **movimientos** el texto “APERTURA por Laura con base 200000” y mostrar un mensaje de confirmación.
5. Al final, mostrar el estado de la caja (ABIERTA o CERRADA) usando el operador ternario, y la lista de movimientos.

Resultado esperado con los datos iniciales

Caja abierta por Laura con una base de 200000

Estado de la caja: ABIERTA

Movimientos: [APERTURA por Laura con base 200000]

Pruebe también: rol = 'mesero'; baseInicial = -5000; baseInicial = 10000001; cajaAbierta = true. Cada caso debe mostrar su mensaje de error y el estado CERRADA (o ABIERTA en el último).

Ejercicio 2 · Registrar la venta de una mesa

Situación: el mesero tomó el pedido de la mesa 4. Hay que calcular la cuenta, agregar la propina voluntaria y registrar el pago. Uno de los productos que anotó el mesero no existe en el menú.

Datos iniciales

```
void main() {
  Map<String, int> menu = {
    'Ajiaco': 28000,
    'Bandeja paisa': 32000,
    'Sancocho': 26000,
    'Limonada': 6000,
    'Jugo natural': 7000,
    'Gaseosa': 5000,
    'Postre del día': 9000,
  };

  Map<String, int> pedido = {
    'Bandeja paisa': 2,
    'Limonada': 3,
    'Postre del día': 1,
    'Cazuela': 1,                  // no está en el menú
  };
  bool incluirPropina = true;      // propina voluntaria del 10 %
  String medioPago = 'efectivo';   // 'efectivo' o 'debito'
  int valorRecibido = 110000;      // solo para efectivo
  String? referencia;              // solo para débito

  // Escriba aquí su código
}
```

Lo que debe hacer:

1. Recorrer el pedido. Si un producto no existe en el menú, mostrar un aviso y **saltarlo** con continue.
2. Mostrar cada línea: producto, cantidad y valor (precio × cantidad), y acumular el subtotal.
3. Calcular la propina (10 % del subtotal, sin decimales) solo si incluirPropina es true. Usen el operador ternario.
4. Mostrar subtotal, propina y total.
5. Con un switch sobre medioPago: en **efectivo**, mostrar el cambio o cuánto falta si el dinero no alcanza; en **debito**, mostrar un error si referencia es null o confirmar el pago; en otro caso, “Medio de pago no válido”.

Resultado esperado con los datos iniciales

Bandeja paisa x2 = 64000

Limonada x3 = 18000

Postre del día x1 = 9000

Cazuela: no existe en el menú, se omite

Subtotal: 91000 · Propina: 9100 · TOTAL: 100100

Cambio: 9900

Pruebe también: incluirPropina = false; valorRecibido = 90000; medioPago = 'debito' con y sin referencia (por ejemplo, referencia = 'A-48213'); medioPago = 'bitcoin'.

Ejercicio 3 · Venta con descuento

Situación: el restaurante tiene una política de descuentos. Un grupo celebra un cumpleaños y es cliente frecuente. El sistema debe calcular el descuento sin superar el máximo permitido.

Datos iniciales

```
void main() {
  Map<String, int> menu = {
    'Ajiaco': 28000,
    'Bandeja paisa': 32000,
    'Sancocho': 26000,
    'Limonada': 6000,
    'Jugo natural': 7000,
    'Gaseosa': 5000,
    'Postre del día': 9000,
  };

  Map<String, int> pedido = {
    'Ajiaco': 3,
    'Jugo natural': 4,
    'Postre del día': 2,
    'Bandeja paisa': 1,
  };
  String? cupon = 'CUMPLE';        // puede ser null si no hay cupón
  bool clienteFrecuente = true;
  const int descuentoMaximo = 20;  // porcentaje máximo

  // Escriba aquí su código
}
```

Política de descuentos (se suman entre sí):

Condición	Descuento
Subtotal mayor o igual a 150.000	10 %
Cupón CUMPLE	15 %
Cupón ESTUDIANTE	5 %
Otro cupón	No válido: mostrar un aviso, sin descuento
Cliente frecuente	5 %
Tope	La suma nunca puede superar el 20 %

Lo que debe hacer:

1. Calcular el subtotal del pedido.
2. Acumular el porcentaje de descuento y guardar en una lista **motivos** el texto de cada descuento aplicado.
3. Revisar el cupón solo si no es null. Acepten el cupón en mayúsculas o minúsculas (pista: **cupon.toUpperCase()**).
4. Si el porcentaje supera el tope, avisar y dejarlo en 20 %.
5. Calcular el valor del descuento sin decimales y el total a pagar. Mostrar el detalle y los motivos.

Resultado esperado con los datos iniciales

Subtotal: 162000

El descuento de 30 % supera el máximo; se aplica 20 %

Descuento: 32400 · TOTAL A PAGAR: 129600

Motivos: compra mayor a 150.000 (10 %), cupón CUMPLE (15 %), cliente frecuente (5 %)

Pruebe también: cupon = null; cupon = 'estudiante'; cupon = 'GRATIS'; clienteFrecuente = false; un pedido con subtotal menor a 150.000.

Ejercicio 4 · Devolución de un producto

Situación: en la mesa 7 un cliente devuelve una gaseosa porque estaba en mal estado. Hay que descontarla de la venta. Las devoluciones de más de 30.000 pesos requieren autorización del administrador.

Datos iniciales

```
void main() {
  Map<String, int> menu = {
    'Ajiaco': 28000,
    'Bandeja paisa': 32000,
    'Sancocho': 26000,
    'Limonada': 6000,
    'Jugo natural': 7000,
    'Gaseosa': 5000,
    'Postre del día': 9000,
  };

  Map<String, int> venta = {'Sancocho': 2, 'Gaseosa': 3, 'Postre del día': 2};
  String productoDevuelto = 'Gaseosa';
  int cantidadDevuelta = 1;
  String motivo = 'Producto en mal estado';
  bool autorizadoPorAdmin = false;
  const int limiteSinAutorizacion = 30000;
  List<String> devoluciones = [];

  // Escriba aquí su código
}
```

Lo que debe hacer:

1. Calcular y mostrar el total original de la venta.
2. Validar que el producto haga parte de la venta y que la cantidad a devolver sea mayor que 0 y no supere la cantidad vendida.
3. Calcular el valor de la devolución. Si supera el límite y no hay autorización, mostrar un aviso y no hacer cambios.
4. Si todo está bien: restar la cantidad en el Map; si la cantidad queda en 0, **eliminar** el producto de la venta.
5. Agregar a la lista devoluciones un texto con la cantidad, el producto, el motivo y el valor.
6. Recalcular y mostrar la venta actualizada y el nuevo total.

Resultado esperado con los datos iniciales

Total original: 85000

Devolución registrada: 5000

Venta actualizada: {Sancocho: 2, Gaseosa: 2, Postre del día: 2}

Nuevo total: 80000

Devoluciones: [1 x Gaseosa (Producto en mal estado): 5000]

Pruebe también: devolver 3 gaseosas (el producto debe desaparecer de la venta); devolver 5 gaseosas; devolver un 'Ajiaco'; devolver 2 sancochos (52.000, requiere autorización) con autorizadoPorAdmin en false y en true.

Ejercicio 5 · Jornada completa y cierre de caja

Situación: terminó el día. La caja se abrió con 200.000 pesos, se hicieron 6 ventas con distintos medios de pago y se pagaron dos gastos en efectivo. El cajero contó el efectivo y hay que hacer el arqueo. Recuerden que las ventas con débito y a crédito **no** entran como efectivo a la caja.

Datos iniciales

```
void main() {
  int baseInicial = 200000;

  // Las dos listas van en el mismo orden: la venta i se pagó con el medio i
  List<int> totales = [45000, 120000, 38000, 76000, 54000, 91000];
  List<String> medios = ['efectivo', 'debito', 'efectivo', 'credito', 'efectivo', 'debito'];

  Map<String, int> egresos = {'Compra de hielo': 15000, 'Recarga de gas': 20000};
  int efectivoContado = 300000;

  // Escriba aquí su código
}
```

Lo que debe hacer:

1. Crear un Map **porMedio** con efectivo, debito y credito en 0, y recorrer las ventas con un **for con índice** para acumular el total de cada medio.
2. Calcular el total vendido, la cantidad de ventas, la venta mayor y el promedio por venta (pista: **promedio.toStringAsFixed(0)** lo muestra sin decimales).
3. Sumar los egresos recorriendo los valores del Map.
4. Calcular el efectivo esperado: base + ventas en efectivo - egresos.
5. Comparar con el efectivo contado y mostrar si la caja está **cuadrada**, si hay **sobrante** o si hay **faltante**, y de cuánto.
6. Mostrar el informe de cierre completo.

Resultado esperado con los datos iniciales

Ventas en efectivo: 137000 · débito: 211000 · crédito: 76000

Total vendido: 424000 (6 ventas) · Venta mayor: 120000 · Promedio: 70667

Egresos: 35000

Efectivo esperado: 302000 · Efectivo contado: 300000

Faltante de 2000

Pruebe también: efectivoContado = 302000 (cuadrada) y 305000 (sobrante). Agreguen una venta nueva con **add** en ambas listas antes del cierre y verifiquen que el informe cambie.

Reto: unir los cinco ejercicios

Escriban un solo programa que simule toda la jornada: abrir la caja (ejercicio 1), registrar la venta de la mesa 4 (ejercicio 2), la venta con descuento (ejercicio 3), la devolución (ejercicio 4), guardar el total de cada venta en las listas totales y medios y, por último, hacer el cierre (ejercicio 5).

Criterios de evaluación

Criterio	Peso
El programa se ejecuta sin errores en DartPad	20 %
Con los datos iniciales muestra el resultado esperado	30 %
Maneja correctamente los casos de “Pruebe también”	30 %
Nombres de variables claros, comentarios y orden del código	20 %